

Project Report Format

Implement Client Script & UI Policy (Incident)

Date	30 September 2026
Team ID	SWTID-2026-5468
Project Name	Implement Client Script & UI Policy (Incident)
Team Leader	Mershitha T
Team Member	Nimisha T

1. INTRODUCTION

1.1 Project Overview

Incident records need consistent and accurate data for effective triage, routing and resolution. This project shows how ServiceNow UI Policies and Client Scripts can enforce data integrity on Incident records. The solution makes fields mandatory, auto-populates values, controls field behaviour and prevents record submission when required conditions are not met.

1.2 Purpose

The purpose of the project is to make sure Incidents are created with complete and valid information, improving data quality and consistency across the platform, reporting accuracy and SLA compliance.

2. IDEATION PHASE

2.1 Problem Statement

Relying only on user awareness and manual checks can lead to incomplete, inconsistent or incorrect Incident data, which affects reporting accuracy, SLA compliance and service quality. Two specific problems were defined:

No.	I am	I'm trying to	But	Because	Which makes me feel
PS-1	a ServiceNow administrator / incident manager	ensure every Incident is created with complete, consistent data	High impact incidents can be submitted without an Assigned To value or with a mismatched Urgency	the form relies only on manual checks instead of built-in UI validation	frustrated and worried about SLA breaches and poor reporting
PS-2	a service desk agent	update incident fields quickly and correctly	State can be changed directly from the list view, bypassing form-level rules	list editing skips the checks applied on the form	concerned that records get changed without proper review

2.2 Empathy Map Canvas

User: ServiceNow service desk agent / incident manager.

Says	Thinks	Does	Feels
"Incidents keep coming in with missing details." "I need to know who owns high impact tickets."	Incomplete data hurts triage, routing and SLA compliance. Rules should be enforced by the form, not by memory.	Creates and updates Incidents daily. Edits fields directly in list views to save time.	Frustrated by inconsistent records. Anxious about missed SLAs on High impact incidents.

Pains	Gains
High impact incidents saved without Assigned To. Urgency not matching Impact. State changed from list view without checks.	Mandatory fields enforced automatically. Urgency auto-set for High impact. Cleaner, reliable data for reports.

2.3 Brainstorming

Ideas listed: (1) a UI Policy that makes Assignment group mandatory and Urgency read-only when Impact is High; (2) an onChange script to auto-set Urgency; (3) an onSubmit script to block save when Assigned To is missing; (4) an onCellEdit script to block State changes from the list; (5) a server-side Business Rule as a backup (out of scope for this micro project).

Prioritization: the UI Policy, the onChange script and the onSubmit script were rated high priority; the onCellEdit script medium; the server-side backup low.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User action	System response
Create / open Incident	Opens Incident → Create New or an existing incident	Form loads with the UI Policy ready to run
Set Impact	Selects Impact = High	Assignment group becomes mandatory, Urgency becomes read-only and is set to High with an info message
Save	Clicks Submit / Update	Save is blocked with an error if Assigned To is empty; otherwise the record is saved
Change Impact back	Changes Impact from High to Medium	Restrictions are reversed (Reverse if false) and the record saves
Edit State	Edits State in the list or on the form	List edit is blocked with an alert; form update is saved

3.2 Solution Requirement

FR No.	Functional Requirement	Sub Requirement
FR-1	High Impact Field Control	UI Policy "High Impact Control" with condition Impact is 1 - High; Assignment group mandatory; Reverse if false
FR-2	Urgency Field Control	UI Policy Action: Urgency read-only when Impact is High; field stays visible
FR-3	Automatic Urgency Update	onChange script sets Urgency to High and shows an info message

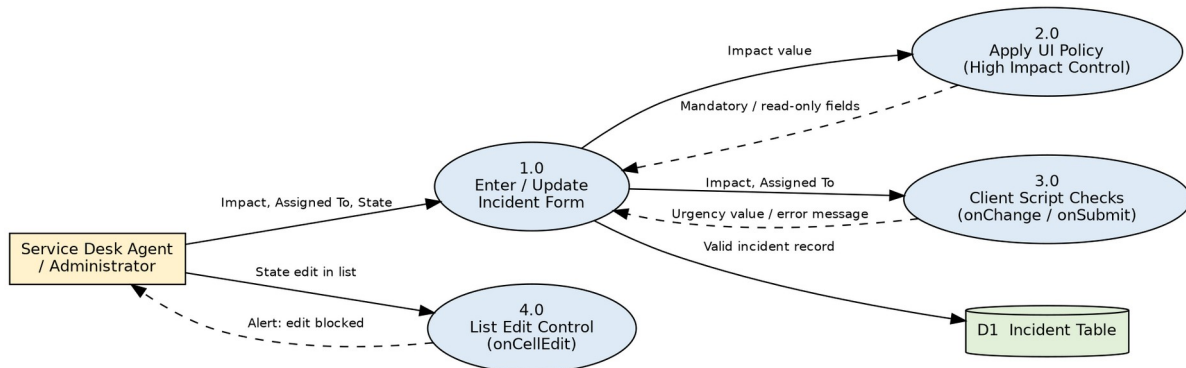
FR-4	Save-time Validation	onSubmit script blocks save when Impact is High and Assigned To is empty, with an error message
FR-5	List Edit Restriction	onCellEdit script blocks State changes from the list and shows an alert
FR-6	Form-based State Update	State can still be changed and saved from the form
FR-7	Reverse Condition Handling	Changing Impact from High to Medium removes the restrictions
FR-8	Testing and Verification	Five test scenarios: mandatory enforcement, successful save, reverse condition, list edit blocking, form update

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	Clear info messages, error box and alert guide the user
NFR-2	Security	Rules are created only with administrative or configuration access; record access follows standard ServiceNow roles
NFR-3	Reliability	Same rules run every time; Reverse if false restores the fields
NFR-4	Performance	Validation runs in the browser with immediate feedback
NFR-5	Availability	Controls are part of the instance configuration
NFR-6	Scalability	Rules are defined once on the Incident table and apply to all records

3.3 Data Flow Diagram



Level 0 data flow diagram

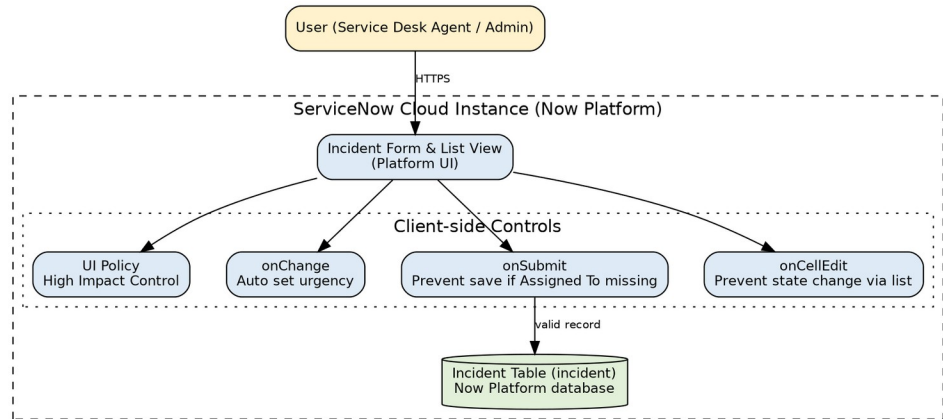


Level 1 data flow diagram

3.4 Technology Stack

Component	Description	Technology
User Interface	Incident form and Incident list view	ServiceNow Platform UI
Application Logic	UI Policy and UI Policy Action; onChange, onSubmit and onCellEdit client scripts	UI Policy, Client Scripts (JavaScript, g_form)
Database	Incident records in the incident table	ServiceNow Now Platform database
Infrastructure	Configured and tested on a ServiceNow instance in the cloud	ServiceNow cloud instance

File storage, external APIs, ML model	Not used	Not applicable
---------------------------------------	----------	----------------



Technical architecture

4. PROJECT DESIGN

4.1 Problem Solution Fit

Problem-Solution fit canvas 2.0 Purpose / Vision: Enforce complete, consistent Incident data on the form

Define CS, fit into CC	1. CUSTOMER SEGMENT(S) CS - Users who create and update Incident records in ServiceNow - Administrators with configuration access who set up UI Policies and Client Scripts	6. CUSTOMER CONSTRAINTS CC - Data entry relies only on user awareness and manual checks - Solution must be lightweight, efficient and easy to implement - Configuration needs administrative or configuration access	5. AVAILABLE SOLUTIONS AS - Relying on user awareness and manual checks - Cons: incomplete, inconsistent or incorrect data can be submitted, which affects reporting accuracy and SLA compliance	Explore AS, differentiate
	2. JOBS-TO-BE-DONE J&P - Enter Incident data consistently and accurately for triage, routing and resolution - Make fields mandatory and auto-populate values - Prevent submission when required conditions are not met	9. PROBLEM ROOT CAUSE RC - Validation depends on manual checks instead of rules enforced at the user interface level - Incidents can be submitted without meeting business rules	7. BEHAVIOUR BE - Users submit Incidents with missing or inconsistent data - State can be edited directly in the list view	
Identify strong TR & EM	3. TRIGGERS TR - Impact is set to High - User saves (submits) the Incident - State is edited in the list view	10. YOUR SOLUTION SL - UI Policy "High Impact Control": when Impact is High, Assignment group is mandatory and Urgency is read-only (Reverse if false) - onChange script: sets Urgency to High when Impact is High - onSubmit script: prevents save if Assigned To is empty for High impact - onCellEdit script: blocks State changes from the list view	8. CHANNELS OF BEHAVIOUR CH 8.1 ONLINE: ServiceNow Incident form and Incident list	Extract online & offline CH of BE
	4. EMOTIONS: BEFORE / AFTER EM - Before: unsure the data is complete and correct - After: confident records meet the business rules		8.2 OFFLINE CH Not applicable: the controls run on the ServiceNow Incident form and list	

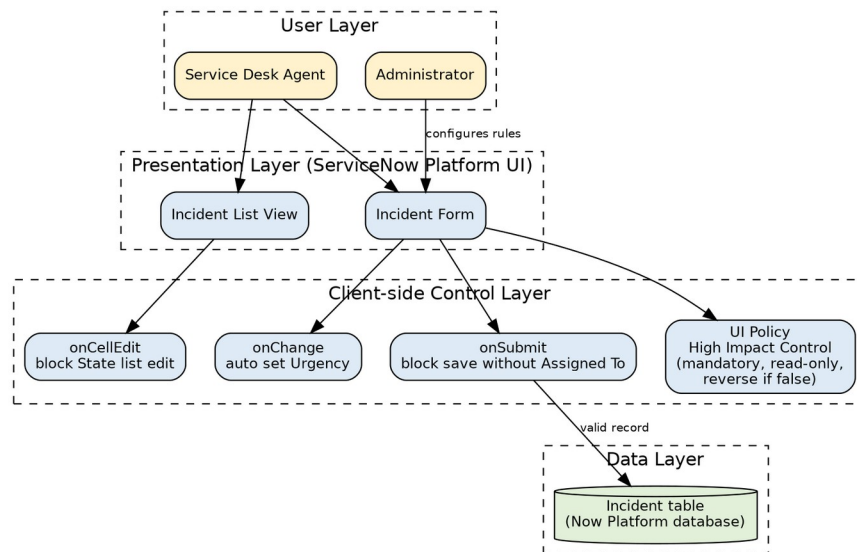
Filled for project: Implement Client Script & UI Policy (Incident) | Team ID: SWTID-2026-5468

Problem-Solution Fit canvas

4.2 Proposed Solution

S.No.	Parameter	Description
1	Problem Statement	Relying on user awareness and manual checks lets incomplete, inconsistent or incorrect Incident data be submitted, affecting reporting accuracy, SLA compliance and service quality.
2	Idea / Solution description	Enforce conditional field behaviour and validation on the Incident form with a UI Policy and three client scripts (onChange, onSubmit, onCellEdit).
3	Novelty / Uniqueness	UI Policies and Client Scripts work together to make fields mandatory, auto-populate values, control behaviour and prevent submission when conditions are not met.
4	Social Impact / Customer Satisfaction	Complete and valid incidents improve data quality, reporting accuracy, SLA compliance and service quality.
5	Business Model (Revenue Model)	Micro project with no revenue model; value is better data quality and service quality.
6	Scalability of the Solution	Lightweight and easy to implement; defined on the Incident table, so it applies to all Incident records.

4.3 Solution Architecture



Solution architecture

5. PROJECT PLANNING & SCHEDULING

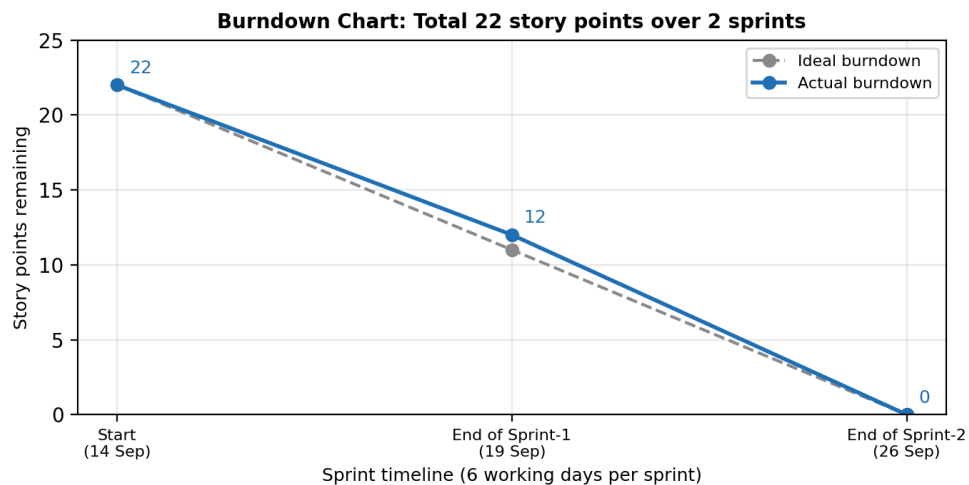
5.1 Project Planning

Sprint	Epic	USN	User Story / Task	Points	Priority
Sprint-1	High Impact Control (UI Policy)	USN-1	Create UI Policy "High Impact Control" with condition Impact is 1 - High	2	High
Sprint-1		USN-2	Make Assignment group mandatory; set Reverse if false	3	High

Sprint-1	Urgency Control (UI Policy Action)	USN-3	Make Urgency read-only when Impact is High	2	High
Sprint-1	Auto Urgency (onChange)	USN-4	Set Urgency to High automatically with an info message	3	High
Sprint-2	Save Validation (onSubmit)	USN-5	Prevent save when Impact is High and Assigned To is empty	3	High
Sprint-2	List Edit Restriction (onCellEdit)	USN-6	Block State edits in the list with an alert	2	Medium
Sprint-2	Testing the Configuration	USN-7	Verify mandatory enforcement and successful save	3	High
Sprint-2		USN-8	Verify the reverse condition	2	Medium
Sprint-2		USN-9	Verify list edit blocking and form-based update	2	Medium

Sprint	Total Story Points	Duration	Sprint Start	Sprint End (Planned)	Points Completed	Release Date (Actual)
Sprint-1	10	6 Days	14 Sep 2026	19 Sep 2026	10	19 Sep 2026
Sprint-2	12	6 Days	21 Sep 2026	26 Sep 2026	12	26 Sep 2026

Velocity = Total story points completed / Number of sprints = 22 / 2 = 11 story points per sprint (about 1.83 per day over a 6-day sprint).



Burndown chart

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

ID	Scenario	Expected result	Result
FT-01	Assignment group mandatory when Impact is High	Assignment group becomes mandatory	Pass
FT-02	Urgency read-only when Impact is High	Urgency visible but read-only	Pass
FT-03	onChange sets Urgency	Urgency = High and info message shown	Pass
FT-04	Save without Assigned To (High impact)	Save blocked with error message	Pass

FT-05	Save with Assigned To filled	Incident saved without errors	Pass
FT-06	Reverse condition (High to Medium)	Restrictions removed; record saves	Pass
FT-07	State edit in list	Alert shown; State unchanged	Pass
FT-08	State update on form	State change saved	Pass
PT-01	Response time of onChange script	Urgency updates immediately without reload	Pass
PT-02	Response time of save validation	Error appears immediately	Pass
PT-03	Repeated Impact changes	Fields apply and revert correctly each time	Pass

User acceptance testing: 7 test cases (TC-001 to TC-007) were executed; 7 passed, 0 failed and no defects were raised.

7. RESULTS

7.1 Output Screenshots

[Insert screenshot] Impact set to High: Assignment group mandatory and Urgency read-only

[Insert screenshot] Urgency set to High with the info message

[Insert screenshot] Save blocked: "Assigned To is mandatory for High impact incidents."

[Insert screenshot] Reverse condition: Impact changed from High to Medium

[Insert screenshot] List edit blocked: State alert

[Insert screenshot] Form-based State update saved

8. ADVANTAGES & DISADVANTAGES

Advantages

- Incidents are created with complete and consistent data.
- Users get immediate feedback on the form without waiting for the server.
- Lightweight and easy to implement using standard ServiceNow configuration; no server-side code.
- Reverse if false removes the restrictions automatically when Impact is no longer High.

Disadvantages

- The controls run only in the user interface; records created by other routes (such as imports or integrations) do not run client scripts.
- The onCellEdit script blocks all State changes from the list, for every user.
- The onSubmit check covers only Impact = High and Assigned To.

9. CONCLUSION

The project shows how UI Policies and Client Scripts work together to enforce dynamic field behaviour, automate updates and prevent incorrect data submission on Incident forms. With the onChange, onSubmit and onCellEdit scripts and the UI Policy actions, fields such as Assigned To and Urgency respond correctly to changes in Impact, keeping the data clean and consistent. The solution is lightweight, efficient and easy to implement.

10. FUTURE SCOPE

- Add a server-side check (for example a Business Rule or Data Policy) as a backup to the client-side rules.
- Extend the rules to more fields, conditions and other tables.
- Allow selected roles to edit State from the list view.

11. APPENDIX

Source Code

onChange Client Script (Impact)

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }

    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

onSubmit Client Script

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == '') {

        g_form.showErrorMessage(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
        );
        return false;
    }
    return true;
}
```

onCellEdit Client Script (State)

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {

    alert('State cannot be updated using list editing. Please open the Incident.');
```

Dataset Link

Not applicable. No dataset is used in this project.

GitHub & Project Demo Link

[Add your GitHub repository / project demo link here]